

Job Authentication in Computation Environment

Dominik Pantůček

Contents

Definitions	2
Motivation and Design Goals	3
Cryptographic Design Rationale	3
Protocol Overview	3
1. Job Creation and Allocation	4
2. Validation Options	4
3. Job Execution and Data Retrieval	4
Data Request Authentication Schemes	4
Option B: Job Authentication via Signed JSON Web Tokens (JWT)	4
Option A: Job Authentication via User-Signed JSON Web Tokens (JWT)	5
References	5

This document specifies a protocol for authenticating jobs executed within computing environments that interact with data repositories built on the CESNET-Invenio software pack. The objective is to ensure that data is provided only when two conditions are met: first, that a request for data originates from a verified computing environment; and second, that a user has explicitly authorized the release of the specific data to that environment. This protocol defines how these requests are authenticated and how providers can verify their authenticity. It does not prescribe the implementation of authorization mechanisms within the repository platform itself.

Definitions

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

Authentication The process of verifying the identity of an entity (e.g., confirming that a system or user is who they claim to be).

Authorization The process of verifying that an authenticated entity has the permissions required to perform a specific action (e.g., confirming that a request is permitted).

Base64URL A binary-to-text encoding scheme used to represent binary data in a text format, specifically designed for use in URLs and filenames (as used in JWTs).

CE (Computing Environment) Any self-contained execution environment. While these are often referred to as “Trusted Computing Environments” (TCEs), within the scope of this protocol, no inherent trust is assumed; all actions performed by or within the environment must be authenticated as part of the security model.

Claim A piece of information asserted about a subject within a JWT (e.g., `jpj`).

Compact Serialization A method of representing a JSON Web Token as a string consisting of three parts: Header, Payload, and Signature, separated by dots (`.`).

Crypt4GH The GA4GH File Encryption Standard, which specifies the format of encrypted containers used to store and transfer genomic data.

Curve25519 An elliptic curve in Montgomery form used as the basis for the X25519 Diffie-Hellman key exchange.

Diffie-Hellman (DH) A key exchange protocol that allows two parties to establish a shared secret over an insecure channel based on the complexity of computing discrete logarithms.

Ed25519 A digital signature system providing 128-bit security with small key sizes and high performance, based on the Curve25519 elliptic curve.

ESK (Environment Secret Key) The secret key used by a Computing Environment to sign JSON Web Tokens (JWTs) when authenticating data requests. This is an environment-level key pair.

EPK (Environment Public Key) The public key corresponding to the ESK, used by a **Repository** to verify the authenticity of JWTs provided by a Computing Environment.

GA4GH Global Alliance for Genomics and Health (<https://www.ga4gh.org/>).

HSM (Hardware Security Module) A physical device providing secure storage for cryptographic keys and enabling the execution of cryptographic operations without exposing private keys to the host environment.

Identity and Access Management (IAM) A framework of policies and technologies used to ensure that the right entities (e.g., users) have the appropriate access to specific resources. In the context of this protocol, an IAM system may be used by a **Repository** to manage, verify, and provide information regarding user identities and their associated public keys (**UPK**).

Job A single computing task executed within a Computing Environment (CE).

Job ID (JobID) A unique identifier assigned to a specific job within a given Computing Environment.

Job PK (JPK) The public key associated with a Job SK, used to identify the cryptographic keys for a specific job.

Job SK (JSK) The secret key associated with a Job PK, used by the CE to sign data and requests related to a specific job.

JWA (JSON Web Algorithms) A specification (RFC 7518, updated by RFC 9864) that defines modern, fully-specified algorithms for JSON Object Signing and Encryption (JOSE).

JWK (JSON Web Key) An auxiliary data structure used to represent and serialize both **Secret Keys** and **Public Keys** as defined in this specification.

JWS (JSON Web Signature) A format specification for signing a designated payload using the algorithms defined in JWA (RFC 7515).

JWT (JSON Web Token) A data structure used to transmit authenticated **Job** identification (RFC 7519).

OKP (Octet Keys for Predefined Curves) A set of algorithms used to represent keys for elliptic curves, specifically Ed25519 and Curve25519, using octets as defined in RFC 8037.

PK (Public Key) A cryptographic data structure derived from a **Secret Key**, representing the identity of an

entity.

Pre-registration The process of provisioning and storing trusted public keys (such as Environment Public Keys or User Public Keys) within the **Repository** configuration to enable the verification of incoming requests.

Repository A particular (data) repository, a specific instance of software system for storing data together with rich descriptive metadata. For practical purposes, we can expect the repository to be implemented using CESNET-Invenio software pack. (Other implementations such as DSpace/CLARIN-DSpace/etc. are completely feasible, though.)

SK (Secret Key) A cryptographic data structure that allows the owner to sign data using specific digital signature algorithms.

Subject (sub) A claim identifying the subject of the JWT, which in this protocol corresponds to the **JobID**.

User A human user who possesses access privileges for both a specific computing environment and a particular **Repository**.

User PK (UPK) The public key that identifies a specific user within the system.

User SK (USK) The secret key used by a user to authenticate and authorize requests related to computing jobs and data access.

X25519 A Diffie-Hellman function based on Curve25519, used to encrypt Crypt4GH containers for specified recipients.

Motivation and Design Goals

The primary motivation for this protocol is to establish a secure framework for authorizing data access between data repositories and computing environments. The goal is to ensure that data is only released when both the environment's identity is verified and the user has explicitly authorized that environment to process specific datasets.

The core requirements driving this protocol are:

- **User-Led Authorization:** Users must be able to authorize a specific Computing Environment (CE) to process specific data. This authorization serves as the permission for the CE to retrieve relevant data from the Repository.
- **Environment Identity:** A Computing Environment must always be uniquely identified by a cryptographic key. Every permitted CE must be explicitly configured within the Repository, establishing a “known-good” list of environments.
- **Data Source Context:** The Repository provides the storage for the data and is responsible for enforcing the policies that govern who can access what data and via which environment.
- **User Key Management:** If user keys are utilized to authorize access (as in Option A), the Repository must be able to recognize these keys. This knowledge may be provided through a direct registry within the Repository or through an external Identity and Access Management (IAM) system.

By addressing these requirements, the protocol ensures that data remains protected throughout its lifecycle – from storage to processing – by verifying that only authorized environments can perform operations on user-approved datasets.

Cryptographic Design Rationale

Data stored in the repository is provided in the **Crypt4GH** container format. When a job within a **Computing Environment (CE)** is granted access to specific data, the repository provides the Crypt4GH container encrypted for the **JPK** associated with that job. Consequently, both users and jobs within the CE must be capable of processing Crypt4GH containers; this requires that their SK/PK pairs be compatible with the X25519 algorithm used in Crypt4GH.

To minimize architectural complexity, this protocol utilizes the same key pairs for both data encryption and request authentication (signing). While signing is performed using the **Ed25519** protocol (based on the Twisted Edwards curve), it is possible to map public points between Montgomery and Twisted Edwards curves as defined in **RFC 7748**. This mapping allows the same set of key pairs to be used for both signing and signature verification, simplifying the management of cryptographic credentials across the platform.

Protocol Overview

The high-level process for authenticating jobs within a computing environment against Repository follows these steps:

1. Job Creation and Allocation

- **Request:** The **User** requests the creation of a new **Job** within the **Computing Environment (CE)**.
- **Allocation:** The **CE** allocates a unique **JobID** and generates a corresponding **JPK/JSK** key pair for that specific job.
- **Delivery:** The **CE** provides the **JobID** and **JPK** to the **User**.

2. Validation Options

The user must validate the **JobID** and **JPK** using one of two methods:

- **Option A (Signing):** The **User** signs the **JobID** and **JPK** using their **USK**.
- **Option B (Separate Channel):** The **User** submits the **JobID** and **JPK** to the **Repository** via a separate communication channel.

3. Job Execution and Data Retrieval

- **Start Request:** The **User** requests the **CE** to begin the job. If **Option A** was selected during validation, the user must include the signed **JobID** and **JPk** as part of this request.
- **Data Authentication:** To retrieve data from **Repository**, the **CE** authenticates its request by providing the following components:
 1. The **JobID**;
 2. The **JPk**;
 3. A signature of the request generated using the **JSK**; and
 4. If **Option A** was used, the signature provided by the **User**.

Data Request Authentication Schemes

This section describes the protocols used to authenticate data requests to **Repositories**. There are two primary schemes available for this purpose, differing in how they verify the relationship between a user, a computing environment (CE), and a specific job.

Both schemes rely on the use of **JSON Web Tokens (JWT)** to transmit authentication metadata. **Option B** serves as the baseline, where the platform verifies that a request originates from a trusted Computing Environment by inspecting JWTs signed by the CE.

Option A extends this model by requiring an additional layer of verification: the user must also provide a signature for the data request. In Option A, the CE acts as a facilitator, using the user’s signature to prove to the **Repository** that the specific job has been explicitly authorized for release by a verified user.

The following sections detail the technical specifications for both schemes.

Option B: Job Authentication via Signed JSON Web Tokens (JWT)

The following section describes the authentication process for data requests using **Option B**. In this method, the **Computing Environment (CE)** provides the **JobID** and **Job PK (JPK)** to the **Repository** by signing a JSON Web Token (JWT). The JWT is transmitted in its compact serialization format.

- 1. JWT Header** The JWT header defines the metadata for the token. It must specify the type as `JWT` and the signature algorithm as `Ed25519`.

```
{"typ": "JWT",  
  "alg": "Ed25519"}
```

Additionally, it may include an optional **JWK (JSON Web Key)** claim containing the **Environment Public Key (EPK)**. This is used to identify which Computing Environment is initiating the request when other identification methods are not available or practical.

```
{ "typ": "JWT",  
  "alg": "Ed25519",  
  "jwk": { "crv": "Ed25519", "kty": "OKP", "x": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"} } }
```

- 2. JWT Payload** The JWT payload contains the identity information for the job. It must include the **JobID** as the **sub** (subject) claim and a custom **ipk** claim containing the Base64URL-encoded **Job PK**.

RFC 7748, Elliptic Curve Methods over Curve25519

RFC 8037, Octet Keys for Predefined Curves (OKP)

RFC 9864, JSON Web Algorithms (JWA)

Crypt4GH, GA4GH File Encryption Standard <http://samtools.github.io/hts-specs/crypt4gh.pdf>

GA4GH Website, <https://www.ga4gh.org/>